IS 584: Deep Learning for Text Analytics
Assignment 3

Volga Sezen

Due by 17.05.2026

In this assignment, you will measure the performance of a sentence embedding model and a hybrid retriever on an information retrieval task. You will log your experiments on weights and investigate ways of enriching queries.

Disclaimer

This is an individual assignment. Please refrain from collaboration. Upon any hurdle, you can contact your TA. It is important you adhere to academic integrity principles.

Regarding AI usage, you must abide by the terms specified in the syllabus. When AI is used to generate ideas, parts of a report or code include AI usage statements.

Do not chase perfect retrieval scores. The goal of this assignment is not to maximise Precision@1 or other metrics, but to demonstrate that you understand why each component works the way it does and can reason about failure cases. A pipeline that achieves modest metrics but is implemented correctly and analysed thoughtfully will be graded more favourably than one with high numbers and shallow discussion.

Deliverables

- An ipython notebook with versions of major packages used (pytorch, sentence_transformers etc.), cell outputs and random seeds visible.
- A .pdf report of main findings of your experiments, supported by relevant figures exported at DPI $\geq$ 300 and comparisons with tables.

Late Submissions

Late submissions will be accepted until the homework quiz date (TBA) with a 10% grade reduction each day.

Background Information

Technical Debt (TD) is a metaphor borrowed from financial debt: it refers to the shortcuts, compromises, and deferred decisions made during software development that accumulate over time and increase future maintenance cost. Just as financial debt incurs interest, technical debt grows harder to resolve the longer it is left unaddressed. Examples include writing code without adequate documentation, skipping data validation steps, or training a model without versioning the dataset. In AI-based software projects specifically, TD manifests across a wide range of activities from data collection and labeling to model deployment and monitoring.

ISO/IEC 5338:2023 is an international standard that defines the lifecycle processes for AI systems. It organizes the development and operation of AI projects into 33 processes under four categories: Agreement Processes, Organizational Project-Enabling Processes, Technical Management Processes, and Technical Processes. Each process describes a set of activities that a team should carry out at a particular stage of the project. For example, *AI Data Engineering* covers data acquisition, preparation, and labeling, while *Quality Management* covers verification and validation. In this assignment, each process is treated as a **document** in the retrieval corpus. You can access the standard by visiting this link. (METU VPN will be necessary to view the document outside the campus.)

Queries are given in the file `td_cases_train_i.csv`. It contains real TD cases collected from AI software projects as part of the ADEP and TÜBİTAK 1001 (Pr. No: 124E655) research projects. (More info here.) Each row represents one instance and has the following structure:

Column	Description
`case_description`	A short natural language description of the technical debt observed in the project (this is your *query*).
`root_cause`	The underlying reason why the debt occurred (also part of the query).
`golden_doc`	The ISO 5338 process section that this case has been mapped to by domain experts (this is your *ground truth*).
`golden_id`	[0-33] integer index of each ISO 5338 section.

Documents provided are summaries of ISO 5338 process subsections generated by Claude Opus 4.7. The model was prompted to add context from standards such as ISO/IEC/IEEE 15288:2023 and ISO/IEC/IEEE 12207:2017, which the original document frequently references. Documents are given as a python dictionary provided in `documents.py` (which can be imported as a module) with keys being process sections (same format as `golden_doc`) and values containing the summaries.

Part 0: Setup

Your task is to retrieve the top-k most relevant documents for each query. Install the `sentence_transformers` package to initiate `all-MiniLM-L6-v2` (a bi-encoder), and embed all of the documents. With this model and the utilities provided in `sentence_transformers`, you will be able to setup a vector database and query it.

Part 1: Data Analysis

In this part you will conduct exploratory data analysis on the **queries**. You are free to choose any method, given the following are addressed in your report:

- **Whether each document is referenced by queries more than once. (4p)**
- **Whether there are documents that are not referenced by any query. (4p)**
- **Whether the queries and their associated documents share keywords or concepts. (7p)**

Part 2: Dense Retrieval

This part involves retrieval with an embedding-based model.

1. Query the vector database with the case and root causes in the training set to obtain document IDs in order of similarity.
2. Calculate the following metrics with respect to the ground truth document IDs:

- Hit@k for $k \in [1, 3, 5]$
- NDGC@k for $k \in [1, 3, 5]$

3. Identify five instances in which the relevant document was ranked below fifth place.

Questions

- **Report performance metrics and comment on their interpretation. (10p)**
- **For failure cases, compare their queries and the contents of the most relevant retrieved document. Speculate on the reason for failure considering the quality of the queries and the documents for this task. (15p)**

Part 3: Hybrid Retrieval

In this section, you will mix keyword searching with dense retrieval to improve the retrieval performance.

1. Define a new retriever with the following changes:
   - Alongside computing cosine similarity to the dense embeddings, a second score is calculated using BM25 based on keywords.
   - `alpha` should be defined as the mixing parameter between normalized BM25 scores and cosine similarities across all documents. Final prediction will be the index of the maximum score in the calculated mix.
2. For different values of `alpha`, calculate your performance metrics and log them somewhere (preferably Weights and Biases, with a link to your public experiment page).

Questions

- **Which alpha value lead to the best performance metric values overall on the training set? (10p)**
- **Comment on the effectiveness of merging keyword based sparse retrieval results. In hindsight, were there any overlaps between queries and documents to be taken advantage of? Please discuss. (15p)**

Part 4: Re-writing Queries

Re-write queries so they match the documents in terms of vocabulary and/or style. You are free to choose your rewriting strategy. Below are two directions to consider, however you are not limited to these, and you are encouraged to combine approaches or invent your own.

- **Query expansion**: Identify domain-specific terms that appear in the correct document but are absent from the original query. Augment the query with these terms in a systematic way. You may only use the training set to find bridging vocabulary.
- **Query reformulation**: Prompt an LLM to rephrase the query in the language of an ISO process standard. The goal is not to add new facts, but to translate practitioner descriptions into formal process-oriented language. Be explicit in your prompt about the target register and audience.

Generate predictions on the training set queries with your hybrid search pipeline with **alpha set to 0** (dense-retrieval only).

1. Compute performance metrics on the training set.
2. Focus on the five instances selected in Part 2, and observe the ranking of retrieved documents.

Question

- **Did the re-writing queries move the correct document in any of your previously failed queries (from Part 2) into the top-5? Please discuss. (10p)**

The following section requires you to run your pipeline on the test set that will be provided on 15.05.2026 via email.

1. Run your hybrid retriever on the test case examples with alpha set to 0 and the best alpha found in part 4. Compute performance metrics as previously instructed.
2. Identify five queries in the test set, in which the relevant document was ranked below fifth place.

Questions

- **Did your pipeline perform similarly on the test set as opposed to the training set? Which metrics showed the greatest difference? (10p)**
- **Did altering the queries help the model retrieve the correct documents? Please discuss by sharing your prompts or any pre/post processing steps. (15p)**

AI Use Statement

Gemini 3.1 Pro has been used to (1) generate code for testing. Claude Opus 4.7 was used to (2) generate summaries of ISO 5338 sections. Regarding changes made, (1) code was tested with different inputs and parameters to ensure assigned work is feasible. Generated code was manually edited to fix bugs. (2) Outputs were fed back into the model to assess alignment with the original document, and small changes were made.